



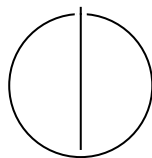
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Author





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

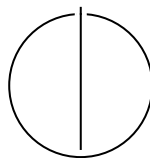
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis, Master's Thesis, ... in Informatics

Thesis title

Titel der Abschlussarbeit

Author:	Author
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis, master's thesis, ... is my own work and I have documented all sources and material used.

Munich, Submission date

Author

Acknowledgments

Abstract

Contents

Acknowledgments	iii
Abstract	iv
1 Introduction	1
2 Background	3
2.1 Register Allocation on non-SSA programs	3
2.1.1 Chaitlin Style Graph Coloring Register Allocation	3
2.1.2 Other Approaches	4
2.2 Register Allocation on SSA programs	5
3 Related Work	6
3.1 Graphcoloring Register Allocation	6
3.2 Linear Scan Register Allocation	6
3.3 Trace Register Allocation	7
4 Approach	8
4.1 Fast Liveness	8
4.2 combining liveness sets and next-use distance	8
4.3 Spilling with next-use distance	8
4.4 Register Allocation with Repairing	8
4.5 VerificationIR (kind of useless)	8
5 Evaluation	9
5.1 Discussion	9
6 Conclusion	10
6.1 Future Work	10
Abbreviations	11
List of Figures	12

Contents

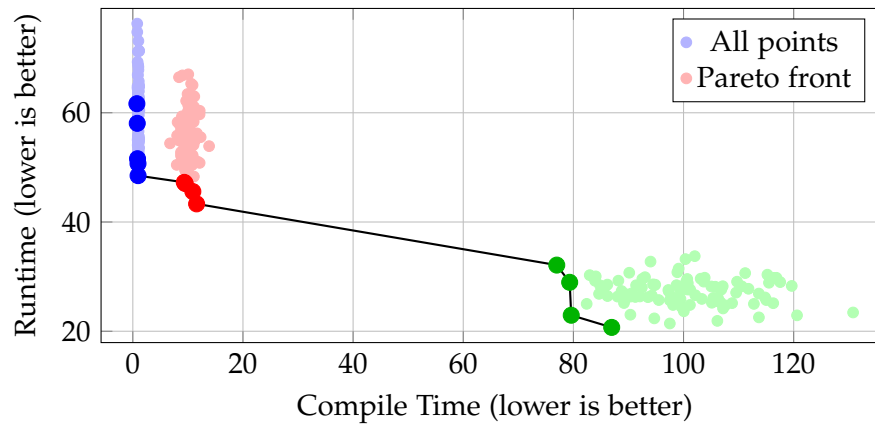
List of Tables	13
Bibliography	14

1 Introduction

- Compile speed is important for some applications like Database systems
- For short running programs, compile time is the dominant factor
- Gains from optimizations are not sufficient to amortize the additional compile time
- For long running programs, runtime is the dominant factor and more expensive optimizations are beneficial
- For most scenarios where a short compile time is wanted, the most interesting metric is the combined compile and runtime of a program [citation](#)
- TPDE [SKE25] is currently the fastest Compiler for LLVM-IR in unoptimized scenarios (-O0) targeting x84-64 and AArch64
- However, transitioning from TPDE to the next optimization level, LLVM [llvm] O1 pipeline is a large step in compile time
- compile time increases by approximately 10 times while delivering only around 5 times runtime improvement (as illustrated in Fig. ??)
- Therefore there is a significant subset of programs in JIT scenarios that are likely to benefit from some level of optimization, but currently lack a fast enough solution

Approach

- fast accurate liveness analysis [cite paper](#)
- utilize SSA form to separate spilling and register allocation phases
- Implement a fast spilling phase without requiring changes to the incoming IR.
- spilling analysis without requiring prior instruction selection phase.
- build a repairing register allocator to address architecture constraints.



Contribution

- Building a simplified spilling mechanism whichi guarantees correctness (even for incorrect liveness data)
- spilling wihtout modifications to IR
- Improving Pareto optimal compilation trade-offs.

Outline

- Background
- Liveness
- Spilling
- Register allocation
- Related work
- Evaluation
- Conclusion

2 Background

In this chapter, we will introduce the basic concepts of register allocation and the particular importance of the SSA form. We then briefly describe the TPDE backend framework and the resulting challenges building register allocation on top of it.

2.1 Register Allocation on non-SSA programs

Register Allocation is the process of assigning values from a program to a limited number of k physical registers. This process is commonly modeled as finding the Graph Coloring of an Interference graph corresponding to the program.

The nodes in an interference graph correspond to the values in a program, and edges between them indicate that the connected values interfere with each other and therefore cannot be in the same register. This nicely corresponds to Graph coloring where connected nodes may not be assigned the same color.

Therefore a valid coloring with at most k colors would be a valid register allocation for the corresponding program on an perfectly regular architecture with at least k registers.

In practice, using such an approach directly is unfeasible as GC is NP-complete in the number of values [GJ90], and any arbitrary IG can be used to construct a program having the same IG [Cha+81]. In addition, deciding how many registers a program requires and deciding whether k registers are sufficient are also NP-complete [Hac07].

Practical RA need to rely on heuristics at all steps, to find a allocation in a reasonable time. We will illustrate this with the seminal work by Chaitin [Cha+81] on the subject. Note that we will assume a theoretical regular architecture for now. We will address architectural constraints and their associated complications later.

2.1.1 Chaitlin Style Graph Coloring Register Allocation

First the Interference Graph itself has to be constructed. This is done by scanning the program and adding edges between values that are live at the same time.

Next, a coalescing heuristic is applied such that the nodes stemming from copy instructions are merged into a single node. Notice that this step might increase the

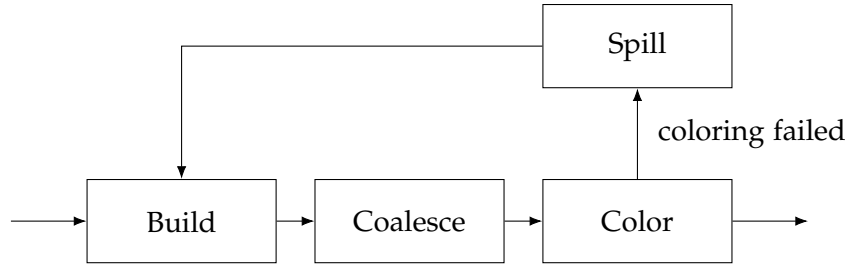


Figure 2.1: Simplified overview of the steps in a graph coloring allocator

number of colors required for the new graph since, the merged nodes may have more edges than previously.

Finally, the main coloring step begins. First all nodes with fewer than k neighbors are removed from the graph and placed into a stack. If the graph is now empty, we move on to the next step. Otherwise, we select a remaining node for spilling and rebuild the IG for the new program after inserting spill code. This process is repeated until the graph is empty.

Now all nodes are on the coloring stack, where we reinsert them into the graph and assign them a color. This will always be possible since they must have fewer than k neighbors. This produces a valid coloring of the final program.

While this approach produces good results in practice, building the Inference Graph is an expensive operation and the iterative nature of the algorithm make it unattractive for use in a JIT compiler [CD06].

While there have been countless improvements to the original algorithm, the basic iterative structure remains the same, prompting different approaches to the problem.

2.1.2 Other Approaches

One alternate approach is to formulate Register Allocation as an Integer Linear Programming problem. While this can produce optimal results it is also NP-complete and the worst case runtimes of several minutes makes them unattractive for JIT compilers [FW02].

The other common approach is to use a linear scan allocator [PEK97]. This approach avoids the expensive IG datastructure in favor of using a total ordering of all program instructions. This ordering is used to determine the single live range of each value, for example starting at instruction i and ending at instruction j ($[i, j]$). The live ranges are then assigned to registers in the order of their interval start.

Once the algorithm runs out of registers, it spills a value, splitting its live range into two parts. The spilled value is typically chosen based upon the furthest live range end.

This approach is simple and non-iterative and therefore attractive for use in a JIT compiler. However, due to the simplified liveness information, the resulting allocation quality is comparatively worse. Especially around control flow such as loops and branches with differing register pressure. Although there have been multiple improvements from the original algorithm, the resulting allocation quality is considered worse than that of graph coloring [WM05].

2.2 Register Allocation on SSA programs

3 Related Work

all papers want instsel -> regalloc

3.1 Graphcoloring Register Allocation

As previously described, graph coloring of interference graphs is equivalent to register allocation [Chaitin1981]. In general this is an NP hard problem [Chaitin1981]. For graphs that are not colorable with k colors, the graph coloring algorithms additionally need to iterate between coloring, spilling and coalescing. Where each spill step requires an expensive recomputation of the interference graph [CD06]. This makes traditional graph coloring register allocation too slow for just in time compilation [CD06]. There are multiple improvements proposed to the original algorithm aiming to improve the code quality study look cgo 2006.

The cost of rebuilding the interference graph after every spilling step can be avoided by approximating the resulting interference graph using additional metadata [CD06]. This sacrifices a small amount of code quality for a significant speedup, which makes it more suitable for just in time compilation. Still compared to other methods, the iterative nature of graph coloring is not desirable when prioritizing compilation speed. We leverage the SSA properties to do all spilling as a preprocessing step, and avoid building the interference graph entirely.

We will consider such SSA-based algorithms such as ours separately as they have different compilation speed properties, even though they are fundamentally based on graph coloring. define graph RA define SSA define k

3.2 Linear Scan Register Allocation

Linear Scan Allocators fundamentally of a single scan over the program, finding liveness and then mapping these live ranges to registers (or memory) cite og linear scan. Although newer research present multiple improvements to the original algorithm cite shiee cgo. Linear Scan Allocators are generally faster than graph coloring due to the single scan and no expensive interference graph construction. However, their code quality is generally worse than graph coloring cite. While some of the limitations

of the original algorithm have been addressed[Wimmer2010], Linear Scan Allocators fundamentally use a non-optimal approach with increasingly elaborate heuristics or post-processing passes to improve code quality[Wimmer2010].

Without a preprocessing step it is not possible for a linear scan to find the optimal spill placement around loops, which is particularly problematic as unnecessary spills and reloads in loops are more costly than outside of loops. Still it is popular choice for JIT Compilers. The original register allocator of TPDE [SKE25] could be considered a heavily modified linear scan allocator.

Our algorithm uses a preprocessing step using SSA properties to find better spill locations, especially around loops. As opposed to linear scan we use a greedy allocator, but with the additional theoretical guarantee that on a unconstrained architecture it will never cause an additional spill.

3.3 Trace Register Allocation

citations For JIT Compilers in particular, trace allocation can be used, which assigns registers on chosen segments incoming control flow, so called traces. This allows for fast algorithm and can use different allocation strategies based on the trace properties. However choosing good traces is non-trivial without runtime profiling information, and the resulting code quality is highly dependent on the trace selection.

As TPDE [SKE25] does not use have access to runtime profiling information, this approach is not suitable for our use case. Our approach allocates across control flow, without runtime information under the assumption that loops will be executed multiple times.

4 Approach

4.1 Fast Liveness

A non iterative liveness analysis on strict SSA.

4.2 combining liveness sets and next-use distance

4.3 Spilling with next-use distance

- Irreducible control flow
- Spill on definition

4.4 Register Allocation with Repairing

- Multi- part challenges

4.5 VerificationIR (kind of useless)

5 Evaluation

5.1 Discussion

6 Conclusion

6.1 Future Work

- Register Aliasing
- Irreducible Control flow
- rest was ich nicht geschafft hab.
-

Abbreviations

List of Figures

2.1	Simplified overview of the steps in a graph coloring allocator	4
-----	--	---

List of Tables

Bibliography

- [CD06] K. Cooper and A. Dasgupta. “Tailoring graph-coloring register allocation for runtime compilation.” In: *International Symposium on Code Generation and Optimization (CGO’06)*. 2006, 11 pp.–49. doi: 10.1109/CGO.2006.35.
- [Cha+81] G. J. Chaitin, M. A. Auslander, A. K. Chandra, J. Cocke, M. E. Hopkins, and P. W. Markstein. “Register allocation via coloring.” In: *Computer Languages* 6.1 (1981), pp. 47–57. issn: 0096-0551. doi: [https://doi.org/10.1016/0096-0551\(81\)90048-5](https://doi.org/10.1016/0096-0551(81)90048-5).
- [FW02] C. Fu and K. Wilken. “A faster optimal register allocator.” In: *35th Annual IEEE/ACM International Symposium on Microarchitecture, 2002. (MICRO-35). Proceedings*. 2002, pp. 245–256. doi: 10.1109/MICRO.2002.1176254.
- [GJ90] M. R. Garey and D. S. Johnson. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. USA: W. H. Freeman & Co., 1990. isbn: 0716710455.
- [Hac07] S. Hack. “Register allocation for programs in SSA Form.” PhD thesis. 2007. 123 pp. isbn: 978-3-86644-180-4. doi: 10.5445/KSP/1000007166.
- [PEK97] M. Poletto, D. R. Engler, and M. F. Kaashoek. “tcc: a system for fast, flexible, and high-level dynamic code generation.” In: *Proceedings of the ACM SIG-PLAN 1997 Conference on Programming Language Design and Implementation*. PLDI ’97. Las Vegas, Nevada, USA: Association for Computing Machinery, 1997, pp. 109–121. isbn: 0897919076. doi: 10.1145/258915.258926.
- [SKE25] T. Schwarz, T. Kamm, and A. Engelke. *TPDE: A Fast Adaptable Compiler Back-End Framework*. 2025. arXiv: 2505.22610 [cs.PL].
- [WM05] C. Wimmer and H. Mössenböck. “Optimized interval splitting in a linear scan register allocator.” In: *Proceedings of the 1st ACM/USENIX International Conference on Virtual Execution Environments*. VEE ’05. Chicago, IL, USA: Association for Computing Machinery, 2005, pp. 132–141. isbn: 1595930477. doi: 10.1145/1064979.1064998.